

Unit 2

Experiment 6 - Scalable Read APIs with Caching & Optimization

Aim:

Create high-performance read APIs that support pagination, sorting, caching, and optimized database queries to power analytics dashboards and post feeds.

Description:

Modern applications such as social media platforms and analytics dashboards require highly efficient read operations to handle large volumes of data. Unlike write-heavy systems, read-heavy systems must prioritize **low latency, scalability, and optimized data retrieval**.

This experiment focuses on designing **high-performance read APIs** using Spring Boot, incorporating techniques such as **pagination, sorting, query optimization, caching, and performance benchmarking**. It also addresses common performance issues like the **N+1 query problem** and introduces strategies to improve backend efficiency.

Theoretical Explanation:

1. Pagination and Sorting

Pagination is used to divide large datasets into manageable chunks, improving performance and usability.

Key Concepts:

- Page number
- Page size
- Sorting criteria

Code Example (Spring Pageable)

```
@GetMapping("/posts")
public Page<Post> getPosts(Pageable pageable) {
    return postRepository.findAll(pageable);
}
```

Explanation:

- Automatically handles pagination
- Supports sorting via query params
- Reduces data load per request

Academic Insight:

Pagination improves time complexity and memory efficiency, making it essential for scalable systems.

2. JPQL and Query Optimization

JPQL allows writing database queries using entity models.

Example

```
@Query("SELECT p FROM Post p WHERE p.author.id = :authorId")
List<Post> findById(Long authorId);
```

Explanation:

- Abstracts database layer
- Improves maintainability
- Optimizes query execution

Academic Insight:

Query optimization aligns with database performance tuning, minimizing execution cost.

3. N+1 Problem and JOIN FETCH

Problem: Fetching related entities results in multiple queries:

1 query → fetch posts
N queries → fetch related data

Solution: JOIN FETCH

```
@Query("SELECT p FROM Post p JOIN FETCH p.comments")
List<Post> findAllWithComments();
```

Explanation:

- Fetches related data in a single query
- Reduces database round trips

Academic Insight:

This improves **query efficiency** and reduces **network overhead**, critical in high-scale systems.

4. Caching with Ehcache

Caching stores frequently accessed data in memory.

Code Example

```
@Cacheable("posts")
public List<Post> getAllPosts() {
    return postRepository.findAll();
}
```

Explanation:

- First request → database hit
- Subsequent requests → cache hit

Academic Insight:

Caching reduces **latency** and improves **throughput**, widely used in distributed systems.

5. Native SQL Queries

Used for complex or performance-critical queries.

Example

```
@Query(value = "SELECT * FROM posts ORDER BY likes DESC LIMIT 5",  
      nativeQuery = true)  
List<Post> findTopPosts();
```

Explanation:

- Direct database execution
- Better performance for complex operations

Academic Insight:

Native queries provide fine-grained control over execution plans.

6. Performance Benchmarking

Benchmarking measures API performance under load.

Using Apache JMeter

Key metrics:

- Response time
- Throughput
- Error rate

Academic Insight:

Benchmarking supports **performance evaluation and capacity planning**.

7. Backend Performance Tuning

Techniques:

- Indexing database columns
- Optimizing queries
- Using caching
- Reducing payload size

Academic Insight:

Performance tuning reflects **system optimization principles**, balancing efficiency and scalability.

Code Integration Example

Paginated & Sorted API

```
@GetMapping("/posts")
public Page<Post> getPosts(
    @PageableDefault(size = 10, sort = "createdAt") Pageable pageable) {
    return postRepository.findAll(pageable);
}
```

Caching Analytics Data

```
@Cacheable("analytics")
public Analytics getAnalytics() {
    return analyticsService.computeAnalytics();
}
```

Experiment 6.1: Pagination & Sorting Implementation

Conceptual Focus:

- Efficient data retrieval
- Pagination techniques
- Sorting mechanisms
- API scalability

Implementation Overview:

- Implement paginated GET API
- Add sorting parameters
- Optimize response size

Experiment 6.2: Caching & Query Optimization

Conceptual Focus:

- Query optimization
- N+1 problem resolution
- Caching strategies
- Performance benchmarking

Implementation Overview:

- Use JOIN FETCH
- Implement caching with Ehcache
- Write native queries
- Benchmark performance

Integration Perspective

Pagination and sorting ensure efficient data delivery, while caching and query optimization enhance performance and scalability. Together, they enable the development of **high-performance read APIs** capable of supporting large-scale applications such as dashboards and content feeds.

Assignments

Assignment 1 (Pagination API)

- Implement paginated GET /posts
- Add sorting

Assignment 2 (Fix N+1 Problem)

- Identify issue

- Apply JOIN FETCH

Assignment 3 (Caching)

- Cache analytics data
- Test performance improvement

Assignment 4 (Native Query)

- Write query for top-performing posts

Assignment 5 (Benchmarking)

- Use JMeter
- Compare performance before and after caching

Key Takeaways

- Pagination improves scalability
- Query optimization reduces execution cost
- Caching enhances performance
- Benchmarking ensures system reliability
- Efficient APIs are critical for real-world systems

Future Scope

These concepts can be extended to implement **distributed caching systems (Redis), advanced indexing strategies, and microservices-based data architectures**, enabling highly scalable and real-time data-driven applications.